

Smartphone BI Agent

Model Architectures — Detailed Reference

The internal architecture of every model — gradient-boosted trees (XGBoost), k-nearest-neighbour retrieval, and the random-forest ensemble — including the feature pipeline, encoders, objective functions, hyper-parameters and measured model complexity.

1. Models at a Glance

Pillar	Architecture	Family	Output
1A Segment	XGBoost Classifier	Gradient-boosted decision trees	4-class softmax
1B Similarity	Weighted k-NN	Instance-based retrieval	Nearest phones
2 Demand	XGBoost Regressor	Gradient-boosted decision trees	log-demand → units
3 Segmentation	k-NN Retrieval (k=10)	Instance-based averaging	Demographic distributions
4 PMF / Failure	Random Forest	Bagged decision trees	Binary probability

> Two model families are used: **tree ensembles** (boosting for P1/P2, bagging for P4) for supervised prediction, and **instance-based k-NN** (P1B/P3) for retrieval where direct supervised learning fails.

2. Shared Feature Pipeline

Every supervised model shares the same pre-processing architecture before the learner:

```
raw spec JSON
   │
   ├─► add_engineered_features()        perf_score, cam_total, pixels,
   │                                    battery_per_inch, ram_x_mem,
   │                                    price_per_ram, value_score
   │
   ├─► OrdinalEncoder (categoricals)    brand_name, processor_brand, os
   │     handle_unknown = use_encoded_value (-1)
   │
   └─► numeric specs passed through (trees need no scaling)
            │
            ▼
        model.predict()
```

Note: tree ensembles are scale-invariant, so numeric features are **not** standardised for P1/P2/P4. Standardisation is applied **only** for the distance-based k-NN models (P1B/P3).

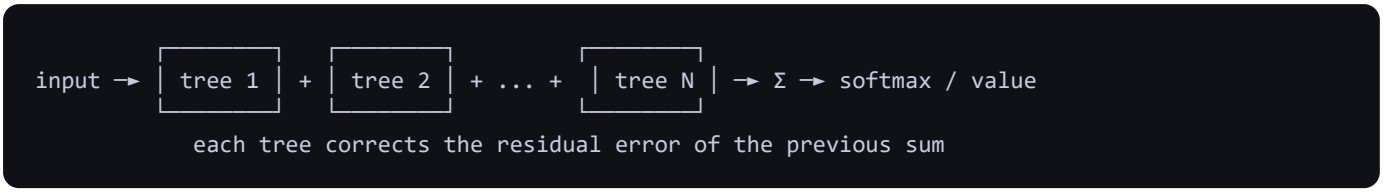
3. XGBoost — Gradient-Boosted Trees (Pillars 1A & 2)

3.1 Architecture

XGBoost builds an **additive ensemble** of CART regression trees. Each new tree is fit to the gradient of the loss left by the current ensemble (functional gradient descent):

```
F_0(x) = base score
F_t(x) = F_(t-1)(x) + η · f_t(x)        η = learning_rate
f_t  = argmin_f  Σ_i [ g_i f(x_i) + ½ h_i f(x_i)² ] + Ω(f)
Ω(f) = γ·T + ½ λ ‖w‖²        (T leaves, leaf weights w)
```

g_i, h_i are the first/second-order gradients of the loss. Classification uses softmax (multi:softprob); regression uses squared error on the log-target.



3.2 Instance — Pillar 1A Segment Classifier

Property	Value
Objective	multi:softprob (4-class)
Boosting rounds (n_estimators)	300
Max tree depth	6
Learning rate (η)	0.1
Subsample	0.9
Input features	21 (18 numeric + 3 categorical)
Classes	4 — budget, flagship, mid-range, premium
Total trees in booster	1,200 (= rounds × classes)

3.3 Instance — Pillar 2 Demand Regressor

Property	Value
Objective	reg:squarederror on ln(1+units)
Boosting rounds	700
Max tree depth	8
Learning rate (η)	0.03
Subsample / colsample_bytree	0.85 / 0.8
min_child_weight	3
Input features	32 (27 numeric + 4 categorical + brand prior)
Total trees	700
Brand-prior lookup size	43 brands

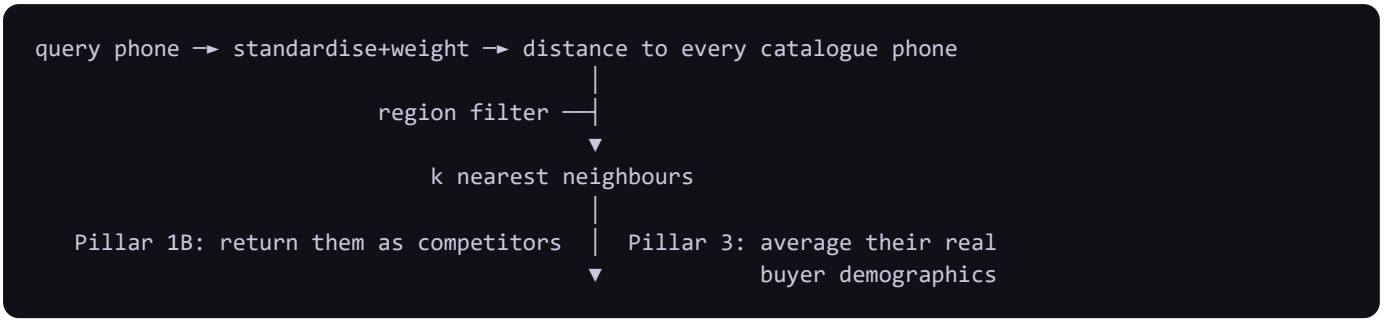
Why deeper + slower for demand. Demand is noisier than segment, so P2 uses a lower learning rate (0.03) with more rounds (700) and deeper trees (8) plus column subsampling to regularise — versus the shallow, fast classifier (depth 6, η 0.1, 300 rounds).

4. k-Nearest Neighbours — Retrieval (Pillars 1B & 3)

4.1 Architecture

k-NN is **non-parametric / instance-based**: there is no trained weight matrix. The "model" is the standardised catalogue itself plus a distance function.

```
1. price' = ln(1 + price)                          (log-scale skewed feature)
2. z_i = (x_i − μ_i) / σ_i                         (standardise each feature)
3. z_price ×= 2.5                                  (up-weight price — defines phone "type")
4. d(a,b) = √ Σ_i (z_a,i − z_b,i)²                 (weighted Euclidean)
5. retrieve k nearest within the same region
```



Property	Value
Reference catalogue (P1B)	5,801 phone×region rows
Distance features	12
Price weight	2.5×
Neighbours (P3)	k = 10, region-aware
Aggregation (P3)	mean of neighbours' distributions, renormalised to 100%

Distance features: price_usd, num_cores, processor_speed, battery_capacity, ram_capacity, internal_memory, screen_size, refresh_rate, primary_camera_rear, 5g_or_not, resolution_height, resolution_width

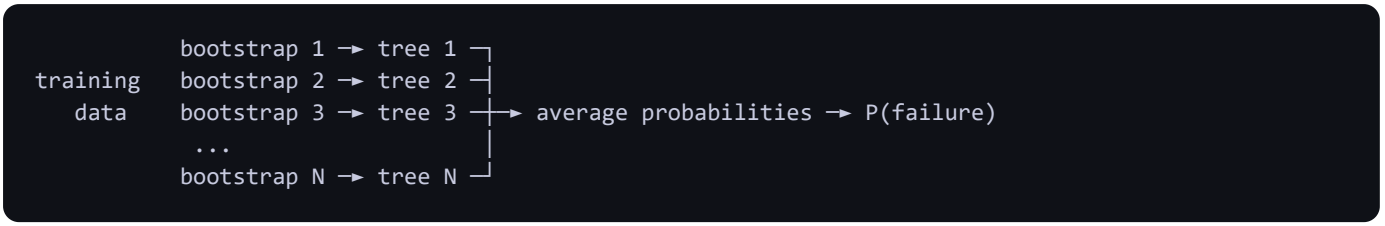
Why k-NN for demographics. A direct specs→demographics regressor scores $R^2 \approx 0$. Retrieval works because "who buys this kind of phone" is best answered by the real buyers of similar phones, not by a parametric function of the specs.

5. Random Forest — Bagged Trees (Pillar 4)

5.1 Architecture

A Random Forest trains many decision trees independently on **bootstrap samples** with a **random feature subset** at each split, then averages their class probabilities (bagging). This decorrelates the trees and reduces variance.

```
P(failure | x) = (1/N) Σ_t  P_t(failure | x)        N = number of trees
each tree t:  bootstrap rows  +  random √p features per split
class_weight = "balanced"  → re-weights the 25% failure class
```



Property	Value
Trees (n_estimators)	500
Max depth (cap)	12
Measured avg depth	11.5
min_samples_leaf	3
Class weight	balanced (handles 25% failure base rate)
Input features	21 (18 numeric + 3 categorical)
Total nodes across forest	64,772
Avg nodes / tree	130

Why a forest, not boosting, for PMF. The constructed failure label is noisier and the dataset smaller (one row per phone), so bagging's variance reduction and the `class_weight="balanced"` option are a better fit than boosting, which could over-fit the quartile cut-off.

6. Training Architecture

```
GroupShuffleSplit(test_size=0.2, random_state=42)
group key = brand_name + "__" + model        →  a phone is wholly in train OR test
leak-free priors (P2 brand prior) fit on the TRAIN fold only, mapped to both folds
cold-start rule: P2 & P4 never receive post-launch columns as features
```

This split is the central guard against inflated metrics: random row-splitting would place the same phone in both folds and let the trees memorise it.

7. Inference Architecture (single request)

```
spec JSON
   │
   ├─ P1A: encode → XGBoost classifier → softmax → segment + confidence
   ├─ P1B: standardise+weight → k-NN → nearest phones (competitors)
   ├─ P3 : k-NN neighbours → average demographics → age/income/gender/RFM
   ├─ P2 : engineer+encode+brand prior → XGBoost reg → expm1 → 6-month curve
   └─ P4 : encode → Random Forest → P(failure) → PMF + GO/CAUTION/RECONSIDER
                                  │
                                  ▼
                  closed-form CLV + price optimiser + web news
                                  │
                                  ▼
                        unified JSON dashboard
```

8. Model Complexity Summary

Model	Type	Trees / k	Depth	Features
P1A Segment	XGBoost clf	1,200 trees	6	21
P2 Demand	XGBoost reg	700 trees	8	32
P4 PMF	Random Forest	500 trees / 64,772 nodes	~11.5	21
P1B Similarity	k-NN	5,801 refs	—	12
P3 Demographics	k-NN	k = 10	—	12

All facts introspected live from models/*.joblib by src/generate_architecture_html.py.